

Supply Chain Data Analysis Report

SQL Data Cleaning and View Creation

The following script initializes the environment and creates a cleaned view of the supply chain data, standardizing date formats and calculating key performance metrics such as lead time variance.

SQL

```
USE dataco_db;

CREATE OR REPLACE VIEW vw_supply_chain_clean AS
SELECT
    order_id,
    order_item_id,
    customer_id,
    category_id,
    STR_TO_DATE(order_date, '%c/%e/%Y %H:%i') AS order_datetime,
    CAST(STR_TO_DATE(order_date, '%c/%e/%Y %H:%i') AS DATE) AS
order_date,
    STR_TO_DATE(shipping_date, '%c/%e/%Y %H:%i') AS
shipping_datetime,
    shipping_mode,
    days_for_shipping_real,
    days_for_shipment_scheduled,
    (days_for_shipping_real - days_for_shipment_scheduled) AS
lead_time_variance_days,
    delivery_status,
    late_delivery_risk,
    market,
    order_region,
    order_country,
    order_city,
    customer_segment,
    customer_country,
    customer_city,
    category_name,
    product_name,
    product_price,
    order_item_quantity,
```

```

sales AS gross_sales_usd,
order_item_discount AS discount_amount_usd,
order_item_discount_rate AS discount_rate,
benefit_per_order AS net_profit_usd,
order_item_profit_ratio AS profit_ratio,
order_status,
CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 END AS
is_loss_making_order
FROM dataco_supply_chain
WHERE order_id IS NOT NULL;

```

Data Preview

A selection of records from the cleaned view to verify data integrity and schema mapping.

SQL

```
SELECT * FROM vw_supply_chain_clean LIMIT 10;
```

The screenshot shows a SQL IDE interface with multiple tabs. The active tab is 'SQL File 4*', which contains the following SQL code:

```

34 order_status,
35 CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 END AS is_loss_making_order
36 FROM dataco_supply_chain
37 WHERE order_id IS NOT NULL;
38
39 • SELECT * FROM vw_supply_chain_clean LIMIT 10;
40

```

Below the SQL editor, the 'Result Grid' is displayed, showing 10 rows of data from the 'vw_supply_chain_clean' view. The columns are: order_id, order_item_id, customer_id, category_id, order_datetime, order_date, shipping_datetime, shipping_mode, and days_for_shipping. The data is as follows:

order_id	order_item_id	customer_id	category_id	order_datetime	order_date	shipping_datetime	shipping_mode	days_for_shipping
77202	180517	20755	73	2018-01-31 22:56:00	2018-01-31	2018-02-03 22:56:00	Standard Class	3
75939	179254	19492	73	2018-01-13 12:27:00	2018-01-13	2018-01-18 12:27:00	Standard Class	5
75938	179253	19491	73	2018-01-13 12:06:00	2018-01-13	2018-01-17 12:06:00	Standard Class	4
75937	179252	19490	73	2018-01-13 11:45:00	2018-01-13	2018-01-16 11:45:00	Standard Class	3
75936	179251	19489	73	2018-01-13 11:24:00	2018-01-13	2018-01-15 11:24:00	Standard Class	2
75935	179250	19488	73	2018-01-13 11:03:00	2018-01-13	2018-01-19 11:03:00	Standard Class	6
75934	179249	19487	73	2018-01-13 10:42:00	2018-01-13	2018-01-15 10:42:00	First Class	2
75933	179248	19486	73	2018-01-13 10:21:00	2018-01-13	2018-01-15 10:21:00	First Class	2
75932	179247	19485	73	2018-01-13 10:00:00	2018-01-13	2018-01-16 10:00:00	Second Class	3
75931	179246	19484	73	2018-01-13 09:39:00	2018-01-13	2018-01-15 09:39:00	First Class	2

At the bottom of the IDE, the 'Output' pane shows the execution log:

#	Time	Action	Message
49	14:02:59	USE dataco_db	0 row(s) affected
50	14:02:59	CREATE OR REPLACE VIEW vw_supply_chain_clean AS SELECT order_id, order_item_id, customer_id, category_id...	0 row(s) affected
51	14:02:59	SELECT * FROM vw_supply_chain_clean LIMIT 10	10 row(s) returned
52	14:04:10	USE dataco_db	0 row(s) affected
53	14:04:10	CREATE OR REPLACE VIEW vw_supply_chain_clean AS SELECT order_id, order_item_id, customer_id, category_id...	0 row(s) affected
54	14:04:11	SELECT * FROM vw_supply_chain_clean LIMIT 10	10 row(s) returned

Analysis Queries

Shipping Performance by Region

This query evaluates late delivery rates and average shipping delays across different regions and shipping modes.

SQL

```
SELECT
    order_region,
    shipping_mode,
    COUNT(order_id) AS total_orders,
    ROUND(AVG(days_for_shipping_real), 2) AS
avg_actual_shipping_days,
    ROUND(AVG(days_for_shipment_scheduled), 2) AS
avg_scheduled_shipping_days,
    ROUND(AVG(days_for_shipping_real -
days_for_shipment_scheduled), 2) AS avg_lead_time_variance_days,
    ROUND(SUM(late_delivery_risk) / COUNT(order_id) * 100, 2) AS
late_delivery_rate_pct
FROM dataco_supply_chain
GROUP BY order_region, shipping_mode
ORDER BY late_delivery_rate_pct DESC
LIMIT 15;
```

Profitability Analysis by Category

Identifies product categories with the highest volume of loss-making orders and calculates the direct financial impact.

SQL

```
SELECT
    category_name,
    COUNT(order_id) AS total_orders,
    SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 END) AS
loss_making_orders,
    ROUND(SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 END)
/ COUNT(order_id) * 100, 2) AS loss_order_pct,
    ROUND(SUM(CASE WHEN benefit_per_order < 0 THEN
benefit_per_order ELSE 0 END), 2) AS total_direct_losses_usd,
```

```
ROUND(SUM(sales), 2) AS total_sales_usd
FROM dataco_supply_chain
GROUP BY category_name
HAVING total_direct_losses_usd < 0
ORDER BY total_direct_losses_usd ASC
LIMIT 10;
```

Financial Impact of Fraud and Cancellations

Analyzes the distribution of sales impact based on high-risk order statuses across different global markets.

```
SQL
SELECT
    market,
    order_status,
    COUNT(order_id) AS order_count,
    ROUND (SUM(sales), 2) AS impact_sales_usd
From dataco_supply_chain
WHERE order_status IN('SUSPECTED FRAUD', 'CANCELED',
'PENDING_PAYMENT')
GROUP BY market, order_status
ORDER BY impact_sales_usd DESC;
```

SQL File 8*
SQL File 4*
SQL File 3*
SQL File 5*
SQL File 6*

Limit to 50000 rows

```

34     market,
35     order_status,
36     COUNT(order_id) AS order_count,
37     ROUND (SUM(sales), 2) AS impact_sales_usd
38 From dataco_supply_chain
39 WHERE order_status IN('SUSPECTED_FRAUD', 'CANCELED', 'PENDING_PAYMENT')
40 GROUP BY market, order_status

```

Result Grid
Filter Rows:
Export:
Wrap Cell Content:
Fetch rows:

order_region	shipping_mode	total_orders	avg_actual_shipping_days	avg_scheduled_shipping_days	avg_lead_time_variance_days	late_delivery
Central Asia	First Class	63	2.00	1.00	1.00	100.00
East Africa	First Class	280	2.00	1.00	1.00	98.93
Southern Africa	First Class	198	2.00	1.00	1.00	98.48
West Africa	First Class	463	2.00	1.00	1.00	97.84
Southern Europe	First Class	1504	2.00	1.00	1.00	97.21
North Africa	First Class	486	2.00	1.00	1.00	97.12
South Asia	First Class	1283	2.00	1.00	1.00	96.65
US Center	First Class	986	2.00	1.00	1.00	96.45
East of USA	First Class	1089	2.00	1.00	1.00	96.42
Eastern Europe	First Class	658	2.00	1.00	1.00	96.05
Northern Europe	First Class	1418	2.00	1.00	1.00	95.98
Central America	First Class	4382	2.00	1.00	1.00	95.50
Southeast Asia	First Class	1552	2.00	1.00	1.00	95.49
West Asia	First Class	994	2.00	1.00	1.00	95.27
Western Europe	First Class	4312	2.00	1.00	1.00	95.25

Result Grid
Form Editor
Field Types
Query Stats
Execution Plan

Result 8
Result 9
Result 10
Read Only

Output

Action Output

#	Time	Action	Message
55	14:06:12	USE dataco_db	0 row(s) affected
56	14:06:12	SELECT order_region, shipping_mode, COUNT(order_id) AS total_orders, ROUND(AVG(days_for_shipping_real), 2) AS ...	15 row(s) returned
57	14:06:13	USE dataco_db	0 row(s) affected
58	14:06:13	SELECT category_name, COUNT(order_id) AS total_orders, SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 ...	10 row(s) returned
59	14:06:13	USE dataco_db	0 row(s) affected
60	14:06:13	SELECT market, order_status, COUNT(order_id) AS order_count, ROUND (SUM(sales), 2) AS impact_sales_usd From d...	10 row(s) returned

SQL File 8*
SQL File 4*
SQL File 3* x
SQL File 5*
SQL File 6*

Limit to 50000 rows

```

34     market,
35     order_status,
36     COUNT(order_id) AS order_count,
37     ROUND (SUM(sales), 2) AS impact_sales_usd
38 From dataco_supply_chain
39 WHERE order_status IN('SUSPECTED FRAUD', 'CANCELED', 'PENDING_PAYMENT')
40 GROUP BY market, order_status

```

Result Grid
Filter Rows:
Export:
Wrap Cell Content:
Fetch rows:

	category_name	total_orders	loss_making_orders	loss_order_pct	total_direct_losses_usd	total_sales_usd
▶	Fishing	17325	3209	18.52	-728570.95	6929653.50
	Cleats	24551	4590	18.70	-452594.21	4431942.66
	Camping & Hiking	13729	2590	18.87	-443082.23	4118425.42
	Cardio Equipment	12487	2332	18.68	-402647.26	3694843.20
	Water Sports	15540	2924	18.82	-334569.63	3113844.60
	Women's Apparel	21035	3923	18.65	-323772.87	3147800.00
	Men's Footwear	22246	4169	18.74	-309269.70	2891757.54
	Indoor/Outdoor Games	19298	3617	18.74	-298637.48	2888993.94
	Shop By Sport	10984	2154	19.61	-144310.56	1309522.02
	Computers	442	73	16.52	-74967.13	663000.00

Result Grid
Form Editor
Field Types
Query Stats
Execution Plan

Result 8
Result 9 x
Result 10
Read Only

Output

Action Output

#	Time	Action	Message
✓ 55	14:06:12	USE dataco_db	0 row(s) affected
✓ 56	14:06:12	SELECT order_region, shipping_mode, COUNT(order_id) AS total_orders, ROUND(AVG(days_for_shipping_real), 2) AS ...	15 row(s) returned
✓ 57	14:06:13	USE dataco_db	0 row(s) affected
✓ 58	14:06:13	SELECT category_name, COUNT(order_id) AS total_orders, SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 ...	10 row(s) returned
✓ 59	14:06:13	USE dataco_db	0 row(s) affected
✓ 60	14:06:13	SELECT market, order_status, COUNT(order_id) AS order_count, ROUND (SUM(sales), 2) AS impact_sales_usd From d...	10 row(s) returned

SQL File 8* SQL File 4* SQL File 3* x SQL File 5* SQL File 6*

Limit to 50000 rows

```

34 market,
35 order_status,
36 COUNT(order_id) AS order_count,
37 ROUND (SUM(sales), 2) AS impact_sales_usd
38 From dataco_supply_chain
39 WHERE order_status IN('SUSPECTED_FRAUD', 'CANCELED', 'PENDING_PAYMENT')
40 GROUP BY market, order_status

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

market	order_status	order_count	impact_sales_usd
Europe	PENDING_PAYMENT	11213	2396945.04
LATAM	PENDING_PAYMENT	11181	2246282.32
Pacific Asia	PENDING_PAYMENT	9458	1877364.31
USCA	PENDING_PAYMENT	5442	1074588.68
Africa	PENDING_PAYMENT	2538	511517.05
LATAM	CANCELED	1103	216113.77
Europe	CANCELED	990	214675.55
Pacific Asia	CANCELED	806	159293.16
USCA	CANCELED	570	111751.99
Africa	CANCELED	223	42535.92

Result 8 Result 9 Result 10 x Read Only

Output

Action Output

#	Time	Action	Message
✓ 55	14:06:12	USE dataco_db	0 row(s) affected
✓ 56	14:06:12	SELECT order_region, shipping_mode, COUNT(order_id) AS total_orders, ROUND(AVG(days_for_shipping_real), 2) AS ...	15 row(s) returned
✓ 57	14:06:13	USE dataco_db	0 row(s) affected
✓ 58	14:06:13	SELECT category_name, COUNT(order_id) AS total_orders, SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 ...	10 row(s) returned
✓ 59	14:06:13	USE dataco_db	0 row(s) affected
✓ 60	14:06:13	SELECT market, order_status, COUNT(order_id) AS order_count, ROUND (SUM(sales), 2) AS impact_sales_usd From d...	10 row(s) returned

Large Dataset Batching Queries

Batch Export: Records 1 to 90,000

SQL

```
SELECT * FROM vw_supply_chain_clean LIMIT 0, 90000;
```

SQL File 8* SQL File 4* SQL File 3* SQL File 5* x SQL File 6*

Limit to 50000 rows

1 • `SELECT * FROM vw_supply_chain_clean LIMIT 0, 90000;`

Result Grid

	order_id	order_item_id	customer_id	category_id	order_datetime	order_date	shipping_datetime	shipping_mode	days_for_shipping
▶	77202	180517	20755	73	2018-01-31 22:56:00	2018-01-31	2018-02-03 22:56:00	Standard Class	3
	75939	179254	19492	73	2018-01-13 12:27:00	2018-01-13	2018-01-18 12:27:00	Standard Class	5
	75938	179253	19491	73	2018-01-13 12:06:00	2018-01-13	2018-01-17 12:06:00	Standard Class	4
	75937	179252	19490	73	2018-01-13 11:45:00	2018-01-13	2018-01-16 11:45:00	Standard Class	3
	75936	179251	19489	73	2018-01-13 11:24:00	2018-01-13	2018-01-15 11:24:00	Standard Class	2
	75935	179250	19488	73	2018-01-13 11:03:00	2018-01-13	2018-01-19 11:03:00	Standard Class	6
	75934	179249	19487	73	2018-01-13 10:42:00	2018-01-13	2018-01-15 10:42:00	First Class	2
	75933	179248	19486	73	2018-01-13 10:21:00	2018-01-13	2018-01-15 10:21:00	First Class	2
	75932	179247	19485	73	2018-01-13 10:00:00	2018-01-13	2018-01-16 10:00:00	Second Class	3
	75931	179246	19484	73	2018-01-13 09:39:00	2018-01-13	2018-01-15 09:39:00	First Class	2
	75930	179245	19483	73	2018-01-13 09:18:00	2018-01-13	2018-01-19 09:18:00	Second Class	6
	75929	179244	19482	73	2018-01-13 08:57:00	2018-01-13	2018-01-18 08:57:00	Second Class	5
	75928	179243	19481	73	2018-01-13 08:36:00	2018-01-13	2018-01-17 08:36:00	Second Class	4
	75927	179242	19480	73	2018-01-13 08:15:00	2018-01-13	2018-01-15 08:15:00	First Class	2
	75926	179241	19479	73	2018-01-13 07:54:00	2018-01-13	2018-01-15 07:54:00	First Class	2
	75925	179240	19478	73	2018-01-13 07:33:00	2018-01-13	2018-01-15 07:33:00	First Class	2
	75924	179239	19477	73	2018-01-13 07:12:00	2018-01-13	2018-01-18 07:12:00	Second Class	5
	75923	179238	19476	73	2018-01-13 06:51:00	2018-01-13	2018-01-15 06:51:00	First Class	2
	75922	179237	19475	73	2018-01-13 06:30:00	2018-01-13	2018-01-15 06:30:00	First Class	2
	75921	179236	19474	73	2018-01-13 06:09:00	2018-01-13	2018-01-13 18:09:00	Same Day	0
	75920	179235	19473	73	2018-01-13 05:48:00	2018-01-13	2018-01-13 17:48:00	Same Day	0
	75919	179234	19472	73	2018-01-13 05:27:00	2018-01-13	2018-01-18 05:27:00	Standard Class	5

vw_supply_chain_clean 2 x Read Only

Output

Action Output

#	Time	Action	Message
✓ 56	14:06:12	SELECT order_region, shipping_mode, COUNT(order_id) AS total_orders, ROUND(AVG(days_for_shipping_real), 2) AS ...	15 row(s) returned
✓ 57	14:06:13	USE dataco_db	0 row(s) affected
✓ 58	14:06:13	SELECT category_name, COUNT(order_id) AS total_orders, SUM(CASE WHEN benefit_per_order < 0 THEN 1 ELSE 0 ...	10 row(s) returned
✓ 59	14:06:13	USE dataco_db	0 row(s) affected
✓ 60	14:06:13	SELECT market, order_status, COUNT(order_id) AS order_count, ROUND(SUM(sales), 2) AS impact_sales_usd From d...	10 row(s) returned
✓ 61	14:08:15	SELECT * FROM vw_supply_chain_clean LIMIT 0, 90000	90000 row(s) returned

Batch Export: Records 90,001 to 100,000

SQL

```
SELECT * FROM vw_supply_chain_clean LIMIT 90000, 100000;
```


